Regina Rexx OO

Regina Rexx 3.9.7 with Object-Oriented Extensions

Introduction, Installation and Reference

Release 1 • updated 17 October 2026
based on Regina Rexx 3.9.7 (18 March 2025)

Publication RXO-0001-0

First Edition (October 2026)

This edition applies to Regina Rexx 3.9.7 with the object-oriented extension, Release 1, as published in the repository github.com/mikebeer/REGINA2, and to all later builds until otherwise indicated in new editions.

Regina Rexx is an open-source REXX interpreter distributed under the GNU Library General Public License. The object-oriented extension described here is distributed under the same license. It is not part of an official Regina release.

Open Object Rexx (ooRexx) is maintained by the Rexx Language Association. This extension follows the syntax of ooRexx where it can, but it is a separate implementation and is not affiliated with that project. IBM and Object REXX are names of IBM products, used here only to identify the language this work descends from.

The repository holds the source, the build tools and the regression tests from which the statements in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface ... 1

Chapter 1. Introduction ... 2
- How it works ... 2
- What is included ... 2
- How it was tested ... 3

Chapter 2. Installing Regina Rexx OO ... 4
- What you need ... 4
- Linux ... 4
- Windows ... 4
- macOS ... 4
- Checking the installation ... 5
- Building from source ... 5

Chapter 3. First Steps ... 7
- A program with a class ... 7
- When the OO features are on ... 7
- Where objects become strings ... 8

Chapter 4. Differences from Classic Regina ... 9
- Constructs that were errors ... 9
- Constructs that keep their meaning ... 9
- Behaviour that changes in OO code ... 9
- Pitfalls ... 9
- New error numbers ... 10

Chapter 5. Differences from Open Object Rexx ... 11
- Supported as in ooRexx ... 11
- Additions ... 11
- Different behaviour ... 11
- Not available ... 12

Chapter 6. Language Reference ... 13
- Messages ... 13
- Environment symbols ... 13
- Instructions ... 13
- Blocks ... 14
- Directives ... 15
- Methods ... 15
- Operators ... 16

Chapter 7. Built-in Classes ... 17
- Object ... 17
- Class ... 17
- String ... 17
- Array, Queue and List ... 18
- Directory, StringTable and Table ... 19
- Stem ... 19
- MutableBuffer ... 19
- Supplier ... 20
- Cons: LISP-style lists ... 20
- Block and the collection methods ... 21
- Python: .Python and .PyObject ... 22

Chapter 8. Sample Programs ... 24
- Classes and methods ... 24
- Inheritance and class libraries ... 25
- Collections ... 27
- Operator methods ... 28
- Stems as arguments and results ... 30
- Data structures ... 31
- Dynamic messages ... 33
- A small application ... 35
- LISP-style lists ... 37

Python libraries ... 38
Blocks ... 40
Chapter 9. Restrictions and Known Problems ... 42
Memory ... 42
Language ... 42
Python ... 42
Errors and tracing ... 43
Platforms ... 43
Performance ... 43
Appendix A. Error Messages ... 44
Appendix B. Switches and Environment Variables ... 46
Appendix C. Contents of the Packages ... 47

Preface

This manual introduces the object-oriented extension of Regina Rexx. It tells you how to install the interpreter from a binary package or from source, how the extension differs from classic Regina and from Open Object Rexx, and it describes the new instructions, directives and classes. It ends with sample programs, the restrictions of this release, and the error messages.

It is written for REXX programmers who know classic REXX. Some familiarity with object-oriented programming helps, but each concept is explained where it is first used.

How this manual is organized

- **Chapter 1, Introduction**, says what the extension is and how it keeps classic programs unchanged.
- **Chapter 2, Installing Regina Rexx OO**, covers the Linux, Windows and macOS packages and building from source.
- **Chapter 3, First Steps**, shows a first program and explains when the OO features are switched on.
- **Chapter 4, Differences from Classic Regina**, lists every construct whose meaning depends on the extension.
- **Chapter 5, Differences from Open Object Rexx**, lists what is supported, what behaves differently and what is missing.
- **Chapter 6, Language Reference**, describes messages, instructions, directives, methods and operators.
- **Chapter 7, Built-in Classes**, lists the classes and their methods.
- **Chapter 8, Sample Programs**, prints the sample programs that come with the packages, with their output.
- **Chapter 9, Restrictions and Known Problems**, lists the limitations of this release.
- **Appendix A** lists the new error messages, **Appendix B** the switches and environment variables, **Appendix C** the contents of the packages.

Conventions

Program text, commands and file names are shown in `monospaced type`. Examples of programs and terminal sessions are shown in framed screens; lines you type and the lines the system prints are not marked separately. REXX keywords are shown in uppercase in the text, although REXX itself ignores case.

Chapter 1. Introduction

Regina is a REXX interpreter for Linux, Windows, macOS and many other systems. Regina Rexx OO is Regina 3.9.7 with an optional object-oriented extension in the style of Open Object Rexx (ooRexx). With it, a REXX program can define classes and methods, create objects, send messages to them and use collections such as arrays and directories:

```
acct = .account~new(100)          /* create an object       */
acct~deposit(50)                  /* send it a message      */
say acct~balance                  /* 150                    */
exit

::class Account                   /* the class definition   */
::attribute balance get
::method init
  expose balance
  use arg balance
::method deposit
  expose balance
  use arg amount
  balance = balance + amount
```

Figure 1. A first object-oriented REXX program

The extension has three goals:

- **Existing programs run unchanged.** A program that does not use the new features runs exactly as it does on Regina 3.9.7: same results, same error messages, same TRACE output. The test suite of Regina itself and a set of classic tests check this for every build.
- **Object orientation is an option.** The OO features switch on when a program uses them, or when it asks for them with a marker or a startup switch. A switch can also turn the whole extension off.
- **ooRexx syntax.** Where it can, the extension uses the syntax and the error numbers of ooRexx, so knowledge and many programs carry over in both directions.

How it works

Every new construct was a syntax error in classic Regina, or is recognized only where a program asks for the extension. That is why classic programs cannot change meaning. For example, `a~b` (no blank before the tilde) was an error and is now a message, while `a ~b` (with a blank) still means "a concatenated with NOT b".

Objects are stored in REXX variables as short internal "handle" strings that cannot be typed in a program. Assignment, stems, arguments and RETURN therefore copy references to objects without any change to the variable pool. Wherever a real string is needed, for example in SAY, in a concatenation or as an argument of a built-in function, the object's STRING method is called, so `say acct` prints what the class decides.

The object system starts only when a program sends its first message, uses a directive, or is run with `--oo`. Until then the interpreter takes exactly the code paths of Regina 3.9.7.

What is included

Area	Contents
Messages	`obj~name(args)` and `obj -> name(args)`, cascades `~~` and `->>`, `~name:super`, `obj~name = value`, `coll[i]` and `coll[i] = value`
Directives	`::class`, `::method`, `::attribute`, `::constant`, `::routine`, `::requires`, `::options`
Instructions	`USE ARG`, `EXPOSE`, `DO x OVER`, `DO WITH INDEX i ITEM x OVER`, `DO COUNTER`, `DO LABEL`, `LOOP`
Classes	Object, Class, String, Array (with any number of dimensions), Queue, List, Directory, StringTable, Table, Stem, MutableBuffer, Supplier; `.nil`, `.true`, `.false`, `.environment`, `.local`

Area	Contents
Methods	instance and class methods, attributes, constants, private and abstract methods, UNKNOWN, operator methods, mixins and multiple inheritance
Stems	stems passed by reference with `USE ARG s.`, stems returned with `RETURN x.`
Lists	LISP-style lists: the Cons class, and with `::requires 'lisp'` the functions `cons`, `car`, `cdr`, `list`, `mapcar` and more
Blocks	`{ :x | x * 2 }` and `{ use arg x; ... }`: routines written where they are used, with the collection methods `do`, `select`, `reject`, `collect`, `detect`, `inject`, `count`, `anySatisfy`, `allSatisfy` and `sortWith`
Python	Python modules from Rexx: `.Python~import('statistics')`, NumPy, pandas, json, datetime and more, through a Python interpreter built into Regina
Control	markers `/* OOREGINA */` and `OPTIONS OOREGINA`, switches `--oo` and `--no-oo`, variable `REGINA_OO`
Errors	error numbers 88, 91, 97, 98 and 99 as in ooRexx, in all message languages

Figure 2. What the extension adds

How it was tested

Each build runs four test suites with `tests/run-regression.sh`. The results of this release are:

Suite	What it checks	Result
trip	Regina's own test suite; output must be identical to Regina 3.9.7	23 of 23 identical
classic	classic programs, including programs that use the new words as variables; recorded with Regina 3.9.7	19 of 19 identical
oo	the extension: syntax, classes, collections, errors, loops, stems, switches	46 of 46
samples	the sample programs of Chapter 8	11 of 11

Figure 3. Test suites

The trip and classic suites are also identical to Regina 3.9.7 when the interpreter is run with `--no-oo`. The Linux build is checked with valgrind. The Windows build runs the oo suite under Wine. The macOS build is cross-compiled; see Chapter 9.

Chapter 2. Installing Regina Rexx OO

What you need

System	Package	Notes
Linux (Debian, Ubuntu; x86-64)	`regina-rexx-oo_3.9.7+oo1_amd64.deb`	Replaces the distribution's `regina-rexx` package. Includes the library, `rexx`, `regina`, `rxstack`, `rxqueue`, man pages, samples and this manual.
Windows (64-bit)	`regina-rexx-oo-3.9.7-win64.zip`	A standalone `rexx.exe`; no DLL, no installation.
macOS 11 or later	`regina-rexx-oo-3.9.7-macos.tar.gz`	One universal `rexx` for Apple Silicon and Intel Macs.
Any other system	the source	See "Building from source" below.

Figure 4. Installation packages

The packages are in the `dist` folder of the repository, github.com/mikebeer/REGINA2, together with a file of SHA-256 checksums.

Linux

Install the package with `dpkg`. If the distribution's Regina is installed, it is replaced; remove the package again to go back to it.

```
sudo dpkg -i regina-rexx-oo_3.9.7+oo1_amd64.deb
rexx -v
rexx /usr/share/doc/regina-rexx-oo/examples/01_classes.rexx
```

Figure 5. Installing on Debian or Ubuntu

The message files are installed in `/usr/share/regina-rexx`, the samples in `/usr/share/doc/regina-rexx-oo/examples` and this manual in `/usr/share/doc/regina-rexx-oo`. To remove the package, use `sudo dpkg -r regina-rexx-oo`.

Windows

1. Unpack the zip file into a folder of your choice, for example `C:\Rexx`.
2. Add the folder to the PATH, or call `rexx.exe` with its full path.
3. Run a sample: `rexx demo\oo\01_classes.rexx`.

`rexx.exe` contains the English error messages. For another language, set `REGINA_LANG` (for example `de`) and `REGINA_LANG_DIR` to the folder with the `*.mtb` files.

macOS

Unpack the archive in Terminal. Files downloaded from the internet carry a quarantine mark, and the binary is not notarized by Apple, so remove the mark before the first run:

```
tar xzf regina-rexx-oo-3.9.7-macos.tar.gz
cd regina-rexx-oo-3.9.7-macos
xattr -dr com.apple.quarantine .
./rexx demo/oo/01_classes.rexx
```

Figure 6. Unpacking and running on macOS

Without the `xattr` command, macOS reports that `rexx` "cannot be opened"; you can then also allow it once in System Settings, Privacy & Security. To install it for all users:

```
sudo mkdir -p /usr/local/bin /usr/local/share/regina-rexx
sudo cp rexx /usr/local/bin/
sudo cp *.mtb /usr/local/share/regina-rexx/
```

Figure 7. Installing on macOS

Checking the installation

`rexx -v` prints the version. The version string is the same as that of Regina 3.9.7, so that classic programs which test it keep working. `rexx -h` lists the switches; the switches `--oo` and `--no-oo` show that the extension is present. The program `demo/oo_demo.rexx` checks classic and OO features and prints `ok` for each of them.

```
rexx -v
REXX-Regina_3.9.7 5.00 18 Mar 2025 (64 bit)
rexx -h | grep oo
  --oo, -O              object-oriented keywords on in every program
  --no-oo, -N           object-oriented extension off (classic Regina)
```

Figure 8. Checking the version and the switches

A program can test for the extension itself:

```
signal on syntax name classic
interpret 'ok = .nil~isNil'
say 'OO extension available'
exit
classic: say 'classic Regina, no OO'
```

Figure 9. Testing for the extension in a program

Building from source

You need a C compiler (gcc or clang), make and git. flex 2.6.4 and bison 3.8.2 are needed only if you change the grammar; the generated files are in the repository.

```
git clone https://github.com/mikebeer/REGINA2.git
cd REGINA2
git checkout oo-syntax
./configure
make
tests/run-regression.sh ./rexx
sudo make install
```

Figure 10. Building and installing from source

Build with `make`, not `make -j`: the Makefile of Regina 3.9.7 has a race between parallel jobs. After changing a structure in `mt.h` or `regina_t.h`, or a node type in `defs.h`, run `make clean` first. After changing `lexsrc.l` or `yaccsrc.y`, run `tools/regenerate-parser.sh` and commit the generated files.

Building the packages

Script	Builds	Needs
`tools/build-deb.sh [dir]`	the Debian package; runs the regression tests first	`dpkg-deb`, a C compiler
`tools/build-windows.sh [dir]`	the standalone `rexx.exe` and the zip file	MinGW-w64 (`x86_64-w64-mingw32-gcc`)
`tools/build-macos.sh [dir]`	the universal macOS `rexx` and the tar.gz file	zig (`pip install ziglang`), `lipo` or `llvm-lipo`

Figure 11. Package build scripts

The Windows and macOS builds leave out readline (line editing in interactive trace), the curses functions and GCI, as their scripts are standalone cross builds. On a Mac, `./configure && make` builds the full interpreter instead.

Chapter 3. First Steps

A program with a class

An object-oriented REXX program has two parts. First comes the main program, as in classic REXX. Then come the directives: lines starting with `::` that define classes, methods and routines. The main program ends at the first of these directives, as if it ended with EXIT.

`::requires` and `::options` are different: they can stand anywhere and never end the main program. The natural place for `::requires` is the top of the program, as for `#include` in C:

```
::requires 'shapes.cls'        /* the classes of a library */
say .circle~new(2)~area         /* the main program follows  */
exit
```

Figure 12. A program that starts with ::requires

```
/* greet.rexx */
g = .greeter~new('Hello')
g~greet('world')
say g~count 'greeting(s)'
exit

::class Greeter
::method init
  expose word count
  use arg word
  count = 0
::method greet
  expose word count
  use arg who
  say word',' who'!'
  count = count + 1
::method count
  expose count
  return count
```

Figure 13. greet.rexx

```
rexx greet.rexx
Hello, world!
1 greeting(s)
```

Figure 14. Running greet.rexx

`.greeter` is an environment symbol: a name with a leading dot that refers to the class. Sending it `new` creates an instance and runs its `init` method with the arguments of `new`. `EXPOSE` makes the listed variables the object's own, shared by all its methods. `USE ARG` takes the arguments without converting them to strings, so objects keep their identity.

When the OO features are on

Message operators (`~`, `->`), `[ ]`, environment symbols and directives always work, because they were not valid in classic REXX. A few new keywords are different: `EXPOSE`, `USE ARG`, `LOOP` and the DO options `LABEL`, `COUNTER` and `WITH` used to be ordinary words. A classic program may use `loop` as a variable or `use arg` as a command. These keywords are therefore recognized only where the program asks for the extension:

In the program or at startup	Effect
`/* OOREGINA */`	A comment holding only this word, anywhere in the file: the keywords are on in the whole file.
`OPTIONS OOREGINA` or `OPTIONS 'OOREGINA'`	The same; the word may also be part of a longer option string such as `OPTIONS 'OOREGINA STRICT_ANSI'`.
a `::` directive line	Any program with directives has the keywords on in the whole file.
an earlier `~`, `->` or `[`	From that point in the source on.
INTERPRET	Inside interpreted strings, once the program uses objects.
`rexx --oo` or `REGINA_OO=on`	On in every program, required file and INTERPRET string.
`rexx --no-oo` or `REGINA_OO=off`	The whole extension is off; programs behave exactly as on Regina 3.9.7, and markers are ignored.

Figure 15. Switching the OO keywords on

Plain Regina 3.9.7 ignores both markers (an unknown option is skipped), so a marked classic program still runs there. The markers are found before the program is parsed, so they can be on any line. A marker inside a string or inside a longer comment does not count.

A switch on the command line wins over `REGINA_OO`. The environment variable is meant for programs started through the REXX API (`RexxStart`) and for `#!` scripts.

Where objects become strings

An object is converted to a string, by sending it the STRING message, wherever REXX needs a string:

- in SAY, in concatenations (`||`, blank and abuttal) and in comparisons with a string on the left,
- as an argument of a built-in function such as `length(obj)`,
- in arithmetic, when the object is the right operand,
- as a command, in PUSH and QUEUE, in INTERPRET and in PARSE,
- as the result of the whole program.

Assignment, arguments of your own routines and methods, RETURN, USE ARG and the items of stems and collections keep the object itself. Every class inherits a STRING method from Object that returns a description such as `an Array`; a class can define its own.

Chapter 4. Differences from Classic Regina

This chapter lists everything that can behave differently from Regina 3.9.7. With `--no-oo`, none of it applies.

Constructs that were errors

Construct	Regina 3.9.7	With the extension
`a~b`, `a~b(1)`, `(x)~b`	Error 64, syntax error	Sends message B
`a->b`, `a -> b`	Error 35.1 at `>`	Sends message B
`a~~b`, `a ->> b`	Error 64 / 35.1	Cascade: sends B, the result is A
`a[1]`, `a[i, j]`	Error 13.1, invalid character	Sends `[]`
`::class Foo` (any `::` clause)	Error 20.1	Directive
`1 + obj`, `'41' = obj`	not possible (no objects)	Uses the string value of the object

Figure 16. Constructs that were errors in classic Regina

Constructs that keep their meaning

Construct	Meaning in both
`a ~b`	A concatenated with NOT B (blank before the tilde)
`~b`, `a~=b`, `a~ =b`, `say~b`	NOT and the comparison "not equal"
`a [1]`	Error 13.1: `[` must follow a term directly
`.nil` in a program that uses no objects	The constant symbol `.NIL`
`use arg x`, `expose a`, `loop`	Commands or ordinary words, unless the OO keywords are on
`do counter`, `do label c`, `do with`	Count loops with the variables COUNTER, LABEL and WITH
`do x over 5` (X and OVER empty)	A loop running 5 times, as in classic REXX
`call f st.` in classic code	Passes the string value of `st.`
PARSE VERSION, `rexx -v`	Unchanged version string

Figure 17. Constructs that keep their classic meaning

Behaviour that changes in OO code

- **RETURN of a stem.** In code with the OO keywords on, `RETURN x.` returns a Stem object with a copy of the stem instead of the stem's default value. Its string value is still the default value or the stem name, so `say f()` prints the same as before.
- **Stem assignment from a Stem object.** `r. = f()` copies the returned Stem object into `r.`. In classic REXX the right side is always a string, so this cannot occur there.
- **Message instructions set RESULT.** A message used as an instruction, such as `list~append(x)`, sets the special variable RESULT to the message's result, as CALL does. Do not use a variable named RESULT for your own data in OO code.
- **TRACE.** Messages and directives are traced like other clauses; TRACE I also shows the position counter of DO OVER loops.

Pitfalls

Three classic REXX rules surprise programmers who write object-oriented code for the first time:

- **A string followed by X or B is a hex or binary string.** `'('x', 'y')'` is an error (error 15) because `'('x` is a hex string. Write `'(' || x || ', ' || y || ')'`.

- **Tails are case sensitive.** `s.tail` with `tail = 'count'` sets the tail `count`, while `s.count` reads the tail `COUNT`.
- **A blank before `~` changes the meaning.** `obj ~msg` is "obj concatenated with NOT msg". Write `obj~msg` or `obj -> msg`.

New error numbers

The extension adds the error numbers 88 (invalid argument), 91 (no result object), 97 (method not found), 98 (execution error) and 99 (translation error), with the numbering of ooRexx. Regina 3.9.7 did not use these numbers. Appendix A lists all messages.

Chapter 5. Differences from Open Object Rexx

The extension follows ooRexx syntax, so many ooRexx programs that use classes, methods and the common collections run unchanged. ooRexx is a much larger system, however. This chapter lists what is supported, what behaves differently and what is missing.

Supported as in ooRexx

- Messages with ~, cascades with ~~, `~name:super`, attribute assignment `obj~name = value`, `[]` and `[]=`.
- The directives `::class` (with SUBCLASS, MIXINCLASS and INHERIT), `::method` (CLASS, PRIVATE, ABSTRACT, ATTRIBUTE), `::attribute` (GET, SET, CLASS), `::constant`, `::routine`, `::requires`.
- Arrays with any number of dimensions: `.array~new(3, 4)`, `a[i, j]`, `dimension`, growing on assignment.
- SELF and SUPER, EXPOSE, USE ARG, the UNKNOWN method, operator methods, mixins and multiple inheritance.
- `DO x OVER`, `DO WITH INDEX i ITEM x OVER`, `DO ... FOR` with OVER, `DO COUNTER`, `DO LABEL`, LOOP.
- The error numbers 88, 91, 97, 98 and 99.

Additions

- `->` and `->>` as alternative spellings of ~ and ~~, with blanks allowed around them.
- The markers `OOREGINA` and the switches `--oo` and `--no-oo` (ooRexx has no classic mode).
- LISP-style lists: the Cons class and the built-in `lisp` library.
- Python modules through the built-in Python interpreter (`.Python`, `.PyObject`); ooRexx has no Python bridge.
- Blocks `{ ... }` see and keep the variables of the code that created them (closures), and have the short form `{ :x | expression }`. In ooRexx 5 a source literal `{ ... }` is a Routine without access to those variables. The collection methods `do`, `select`, `collect`, `inject` and the others come from Smalltalk.
- `::requires` anywhere in a program, for example at the top.
- Regina's classic features remain: its built-in functions, OPTIONS, the external queue, ADDRESS WITH and the Regina API.

Different behaviour

Topic	ooRexx	Regina Rexx OO
Memory	Garbage collection frees unused objects	Objects live until the program ends (Chapter 9)
Keywords	Always reserved where they apply	EXPOSE, USE ARG, LOOP and the DO options need a marker, a directive or `--oo` (Chapter 3)
`.nil` and other environment symbols	Always objects	Constant symbols until the object system is active
Stem assignment `r. = stemobject`	r. refers to the same Stem object	r. receives a copy
`a. = b.`	a. refers to the Stem object of b.	Classic REXX: the default value of a. is set
Class names	`~id` returns the name	`~id` returns the name in uppercase for classes defined with `::class`
`::requires` position	Before the other directives; as every directive ends the main program, it stands after the main program	Anywhere: at the top as in C, in the main program or between methods; it never ends the main program

Topic	ooRexx	Regina Rexx OO
Sorting	`sort` uses the Comparable interface; `sortWith`	`sort` orders the string values; no comparators
`::requires` search	Program folder, current folder, REXX_PATH, PATH	Program folder, current folder, REGINA_MACROS; extensions "", .cls, .rex, .rexx
Required files	Only PUBLIC classes and routines are visible to the requiring program	All classes and routines are visible
Error messages	ooRexx texts	Same numbers, Regina texts (English, German)

Figure 18. Behaviour that differs from ooRexx

Not available

- **Concurrency:** GUARD, REPLY, START, the GUARDED and UNGUARDED options (accepted and ignored), and the Message and Monitor classes.
- **Instructions:** RAISE, FORWARD, SIGNAL ON USER, SELECT LABEL, `::resource`.
- **Environment symbols:** `.methods`, `.routines`, `.context`, `.stdout`, `.stdin`, `.error`, `.endofline` and similar.
- **Classes:** Bag, Set, Relation, CircularQueue, IdentityTable, Collection classes as mixins (OrderedCollection and others), Comparable, Comparator, Method, Routine, Package, Stream, File, DateTime, TimeSpan, RegularExpression, RexxContext, WeakReference, Pointer and Buffer.
- **Methods:** many methods of the String class beyond the built-in functions, for example `caselessPos` or `decodeBase64`, and many methods of the collection classes.

Most ooRexx programs that use these features need small changes. Programs that use only classes, methods and the common collections usually run as they are.

Chapter 6. Language Reference

Messages

Form	Meaning
`receiver~name` `receiver~name(arg1, arg2)`	Sends the message NAME to the receiver. The result is the method's return value; a message in an expression must return a result (error 91).
`receiver -> name(args)`	The same as ~. Blanks are allowed around `->`; the characters `-` and > must be adjacent.
`receiver~~name(args)` `receiver ->> name(args)`	Cascade: sends the message and returns the receiver, so several messages can be chained: `list ->> append(1) ->> append(2)`.
`receiver~name:super(args)`	Inside a method: starts the method lookup in the superclass of the method's class.
`receiver~name = value` `receiver~name(i) = value`	Sends `name=` with the value as first argument (and I as second): attribute assignment.
`receiver[i]`, `receiver[i, j]`	Sends `[]` with the indexes. The `[` must follow the receiver without a blank.
`receiver[i] = value`	Sends `[]=` with the value and the indexes.
`receiver~"+"(x)`	A message name can be a string; this is how operator methods are called directly.

Figure 19. Message forms

The receiver can be any term: a variable, a literal (`'abc'~length`), an expression in parentheses or another message. Message terms bind tighter than prefix operators: `-a~size` is `-(a~size)`. A string receiver is an instance of the String class.

A message can be used as an instruction: `obj~display`. Its result, if any, is assigned to the special variable RESULT.

Environment symbols

A symbol with a leading dot followed by a letter, such as `.array` or `.nil`, is an environment symbol. It names a class (built-in or defined with `::class`), or one of these objects:

Symbol	Value
`.nil`	The NIL object, "no value"; `obj~isNil` tests for it. Collections return it for missing items.
`.true`, `.false`	The strings `1` and `0`.
`.environment`, `.local`	A Directory for global values shared by all parts of the program.

Figure 20. Environment objects

An environment symbol that names nothing evaluates to its own name, as a constant symbol does in classic REXX.

Instructions

USE ARG

```
USE ARG name1, name2., ...
```

Assigns the arguments of the routine or method to the variables in order, without converting objects to strings. An omitted argument drops the variable; `arg(n, 'O')` tests for it. A name can be left out to skip an argument: `use arg , second`.

If a name ends with a dot (`s.`) and the caller passed a plain stem (`call f st.`, `f(st.)` or `obj~m(st.)`), `s.` becomes an alias of the caller's stem: the callee reads the caller's values, and everything it sets, drops or adds is

seen by the caller. If the argument is a Stem object, `s.` refers to it. Any other value becomes the default value of `s..`

EXPOSE

```
EXPOSE name1 name2 stem. ...
```

The first instruction of a method. The listed variables are the object's own variables, shared by all methods of the same class for that object. A stem or compound variable can be exposed as well. Variables that are not exposed are local to the method call.

DO and LOOP

All classic forms of DO work as before. The extension adds:

Form	Meaning
`DO x OVER coll`	X takes each item of an Array, List or Queue, each index of a Directory, StringTable or Table, each tail of a stem (`DO t OVER st.`) or of a Stem object. Other objects are asked for `makeArray`.
`DO x OVER coll FOR n`	At most N iterations.
`DO WITH INDEX i ITEM x OVER coll`	Sets the index and the item: the position and element of an Array, the key and value of a Directory or Table, the tail and value of a stem. Either part may be left out, the order is free, FOR may follow. Other objects are asked for `supplier`, then `makeArray`.
`DO COUNTER c ...`	C is 0 at the start and grows by 1 before each pass through the body. Works with every kind of loop.
`DO LABEL name ...`	Names the loop for LEAVE, ITERATE and END. The control variable still works there too. A plain `DO LABEL name` group runs once and can be left with `LEAVE name`.
`LOOP ...`	The same as DO, except that LOOP without repetitor and condition loops until LEAVE.

Figure 21. The new forms of DO

WHILE and UNTIL work with all forms. In an OVER loop, WHILE is tested after the variables have been set to the next item. The collection is copied when the loop starts, so changes to it in the body do not change the loop. LABEL and COUNTER come first, in either order: `DO LABEL outer COUNTER n i = 1 TO 10`.

RETURN with a stem

In code with the OO keywords on, `RETURN x.` returns a new Stem object with a copy of all tails and the default value of `x.`. Assigned to a stem (`r. = f()`), it replaces `r.` with that copy; assigned to a simple variable, it is an object (see Stem in Chapter 7).

Blocks

```
{ clauses }
{ :name1 :name2 ... | expression }
{ | expression }
```

A block is a routine without a name, written as a term in an expression. Its value is a Block object; `b~call(args)` (or `b~value(args)`) runs it with the arguments and returns what it returns. Collection methods such as `select` and the LISP functions call the blocks they are given.

The short forms are the Smalltalk notation: `{ :x :y | x + y }` means `{ use arg x, y; return x + y }`, and `{ | i < 10 }` means `{ return i < 10 }`. The expression ends at the closing brace. In the long form the clauses are separated by `;` or line ends, as in a program; a block can contain labels, loops and other blocks.

Variables in a block	Rule
Variables of the creator	A block sees the variables of the routine or method that evaluated `{ ... }` and can change them, like an internal routine without PROCEDURE. Inside a method that includes SELF and the exposed variables.
Parameters	The names of a first `USE ARG` (and of `:x` in the short form) are the block's own, even when the creator has variables of that name.
New variables	Variables that the creator did not have when the block was called are local to the call, as are RESULT, RC and SIGL.
After the creator has returned	The block keeps the variables the creator had when it returned, and changes them: a block returned by a routine works as a closure (see `counter` in `11_blocks.rexx`).
PROCEDURE	A block that begins with `PROCEDURE [EXPOSE ...]` has only its own variables and the exposed ones, like an internal routine.

Figure 22. Variables in blocks

Labels inside a block belong to the block; a CALL or function call to a name that is not a label of the block finds the labels of the code that created it, so a block can call the program's internal routines. Errors in a block are reported with the line numbers of the program. `.Block~new(source)` makes a block from a string.

```
limit = 100
big = numbers~select({ :x | x > limit })
total = numbers~inject(0, { :sum :x | sum + x })
names~sortWith({ :a :b | length(a) <= length(b) })
numbers~do({ use arg x, i; say i':' x })
```

Figure 23. Blocks

Directives

Directives start with `::` at the beginning of a clause. The main program ends at the first directive, except `::requires` and `::options`, which can stand anywhere: the statements after them continue what came before, the main program or the body of the last method or routine.

Directive	Options and meaning
`::class name`	`SUBCLASS parent` (default Object), `MIXINCLASS parent` (a class whose methods other classes inherit), `INHERIT mixin ...`. `PUBLIC`, `PRIVATE` and `METACLASS` are accepted; in this release all classes of a required file are visible (Chapter 9).
`::method name`	`CLASS` (a class method), `PRIVATE` (only through SELF), `ABSTRACT` (subclasses must define it; calling it is error 98.6), `ATTRIBUTE`. `GUARDED`, `UNGUARDED`, `PROTECTED` and `UNPROTECTED` are accepted and ignored. Operator methods are named in quotes: `::method "+"`.
`::attribute name`	Defines `name` and `name=` for an exposed variable. `GET` or `SET` defines only one of them; `CLASS` and `PRIVATE` as for methods.
`::constant name value`	A method that returns VALUE.
`::routine name`	A routine called with `CALL name` or `name()`. `PUBLIC` and `PRIVATE` are accepted; all routines of a required file are visible.
`::requires 'file'`	Loads the classes and routines of another file and runs its main program (prolog) once. It can stand anywhere, best at the top. All required files are loaded before the program starts, in the order of the `::requires` directives, wherever they stand. The file is searched next to the program, in the current directory and in REGINA_MACROS, with the extensions "", .cls, .rex and .rexx.
`::options ...`	Accepted and ignored; can stand anywhere, like `::requires`.

Figure 24. Directives

Methods

A method body runs like a routine called with PROCEDURE: it has its own variables (except those it EXPOSEs), its own condition traps and the default NUMERIC settings. A SYNTAX error in a method that the method does

not trap is passed to the caller, so `SIGNAL ON SYNTAX` in the caller catches it. Labels in a method or routine belong to that body.

Name	Meaning inside a method
SELF	The receiver of the message.
SUPER	The superclass of the method's class, for `~name:super`.
`arg()`, `arg(n)`	The arguments, as in a routine.
UNKNOWN	A method named UNKNOWN receives every message the class does not define, with the arguments NAME and an Array of the arguments.
INIT	Runs for every new instance, with the arguments of `new`.

Figure 25. Special names in methods

Method lookup searches the class, then its inherited mixins, then its superclasses. Private methods can be called only through SELF.

Operators

If the left operand of an operator is an object, the operator is sent to it as a message: `a + b` is `a~"+"(b)`, and the prefix `-a` is `a~"-"()`. This works for the arithmetic, comparison, logical and concatenation operators. Without an operator method, objects compare by identity (=, ==, \=, <> and the others), and the other operators use the string value. If the left operand is not an object but the right one is, the right operand is used by its string value.

Chapter 7. Built-in Classes

All classes are subclasses of Object. Class methods are sent to the class (`.array~of(1, 2)`), instance methods to an object. Method names are not case sensitive.

Object

The root of all classes.

Method	Meaning
`init`	Called by `new`; does nothing.
`string`, `defaultName`, `objectName`	A description such as `an Array`; `objectName=` sets a name.
`class`	The object's class.
`isA(c)`, `isInstanceOf(c)`	True if the object is an instance of class C or of a subclass.
`= == \= \== <> ><`	Identity comparisons.
`hashCode`, `identityHash`	A number identifying the object.
`isNil`	True only for `.nil`.
`send(name, args...)`, `sendWith(name, array)`	Sends a message whose name is in a variable.
`hasMethod(name)`	True if the object understands NAME.
`request(classid)`	The object itself if it is of that class, otherwise `.nil`.
`copy`	A copy of the object with copies of its variables.

Class

Every class is an object of class Class.

Method	Meaning
`new(args...)`	Creates an instance and runs its `init`.
`id`, `name`	The name of the class.
`superclass`, `superclasses`	The direct superclass, an Array of all direct superclasses.
`subclass(name)`, `mixinclass(name)`	Creates a subclass at run time.
`inherit(mixin)`	Adds a mixin class.
`isSubclassOf(c)`	True if C is a superclass or an inherited mixin.
`queryMixinClass`	True if the class is a mixin class.
`methods`	An Array with the names of the instance methods.

String

Every string is an object of class String. Each built-in string function is also a method, with the string as first argument: `'abc'~length`, `'a-b'~changeStr('-', '+')`, `'hello'~substr(2, 3)`. In addition:

Method	Meaning
`upper`, `lower`	Uppercase, lowercase.
`equals(s)`, `caselessEquals(s)`	Comparison.

Method	Meaning
`startsWith(s)`, `endsWith(s)`, `contains(s)`	And their caseless forms.
`makeArray(separator)`	An Array of the parts; without separator, the lines.
`makeString`, `string`	The string itself.

Array, Queue and List

Sequences indexed by position. Array has holes where no item was stored and can have several dimensions (see below); Queue and List are ordered lists.

Method	Meaning
`of(items...)` (class)	A new collection with these items.
`[] at(i)`, `[]= put(item, i)`	Read and store; a missing item is `.nil`.
`append(item)`, `insert(item, i)`	Add at the end, insert after position I.
`size`, `items`, `isEmpty`, `empty`	Last index, number of items, test, remove all.
`first`, `last`, `firstItem`, `lastItem`, `next(i)`, `previous(i)`	Navigation.
`hasIndex(i)`, `hasItem(x)`, `index(x)`	Tests and search.
`remove(i)`, `removeItem(x)`	Remove by position or by value.
`makeArray`, `allItems`, `allIndexes`, `supplier`, `section(i, n)`	Copies and walking.
`toString(format, separator)`	One string; format `C` joins without line breaks.
Array only: `fill(x)`, `sort`, `delete(i)`	Fill, sort the string values, delete with shift.
Queue only: `push`, `queue`, `pull`, `peek`	Add in front or at the end, take and look at the first item.

Arrays with several dimensions

`.array~new(3, 4)` creates an array with 3 rows and 4 columns; an empty array also takes the number of dimensions of its first item, so `a = .array~new; a[2, 3] = x` makes a two-dimensional array. Every index of `[]`, `at`, `put`, `hasIndex` and `remove` is given separately: `m[i, j]`, `m~put(x, i, j)`. Storing beyond a bound makes that dimension larger; the items keep their indexes.

Method	Meaning
`dimension`	The number of dimensions.
`dimension(n)`	The size of dimension N.
`dimensions`	An Array with the sizes of all dimensions.
`size`, `items`	The number of places (the product of the sizes) and the number of items.
`index(x)`, `allIndexes`, `supplier`	Indexes are Arrays of numbers, for example `2,3` for row 2, column 3.

Figure 26. Methods for arrays with several dimensions

`DO x OVER m` walks the items row by row; `DO WITH INDEX ix ITEM x OVER m` gives the index as an Array: `ix[1]` is the row, `ix[2]` the column. The methods that treat an array as a list (`append`, `insert`, `delete`, `first`, `last`, `next`, `previous`, `section` and `sort`) need a one-dimensional array (error 98.8). The wrong number of indexes is error 98.11.

```
m = .array~new(2, 3)
do i = 1 to 2
```

```
  do j = 1 to 3
    m[i, j] = i * 10 + j
  end
end
say m[2, 3] m~dimension m~dimensions~toString(, 'x')     /* 23 2 2x3 */
m[3, 4] = 'grown'
say m~dimensions~toString(, 'x') m[2, 3]                  /* 3x4 23  */
```

Figure 27. A two-dimensional array

Directory, StringTable and Table

Collections indexed by key. Directory and StringTable use the string value of the key; Table also accepts objects as keys and compares them by identity.

Method	Meaning
`[] at(k)`, `[]= put(item, k)`	Read and store; keys are case sensitive.
`hasIndex(k)`, `hasItem(x)`, `index(x)`, `remove(k)`	Tests, search and removal.
`items`, `isEmpty`, `empty`	Counting and clearing.
`makeArray`, `allIndexes`, `allItems`, `supplier`	Keys, items and walking.
Directory, StringTable: `entry`, `setEntry`, `hasEntry`, `removeEntry`	Access by name, uppercased as in ooRexx.
Directory, StringTable: any other message	`d~city` reads and `d~city = x` sets the entry CITY.

Stem

A stem as an object. `RETURN x.` creates one in OO code; `.stem~new` or `.stem~new(default)` creates an empty one.

Method	Meaning
`[] at(t...)`, `[]= put(v, t...)`	Read and set a tail; several indexes are joined with dots. A missing tail gives the default value or `.nil`.
`remove(t)`, `hasIndex(t)`	Drop and test a tail.
`items`, `isEmpty`, `empty`	Number of tails with a value, test, drop all.
`allIndexes`, `makeArray`, `allItems`, `supplier`	Tails sorted as DO OVER sorts them, values, walking.
`string`	The default value, or the stem's name.

MutableBuffer

A string that can be changed in place.

Method	Meaning
`new(s)`, `append(s...)`, `insert(s, pos)`, `delete(pos, n)`	Build and change; `append` returns the buffer.
`length`, `string`	Length and value.
`substr pos lastPos left right words word countStr verify`	As the built-in functions.

Supplier

Walks index/item pairs, returned by `supplier`.

Method	Meaning
`available`, `index`, `item`, `next`	Test, current index and item, step.
`allIndexes`, `allItems`	The remaining indexes and items.

Cons: LISP-style lists

A LISP list is a chain of cells. Each cell (a Cons) holds two values, the CAR (the item) and the CDR (the rest of the list); the last cell of a list has `.nil` as its CDR, and `.nil` is the empty list. `say` prints a list in LISP notation: `(1 2 3)`, nested lists as `(1 (2 3))`, a cell whose CDR is no list as a dotted pair `(a . b)`, and atoms with blanks in quotes.

Method	Meaning
`new(car, cdr)` (class)	A new cell.
`of(items...)` (class)	A list of the items; without items `.nil`.
`read(string)` (class)	The list written in LISP notation in the string: `( )`, atoms, `"strings"` (`""` for a quote), `(a . b)`, `NIL`, and `;` comments. A syntax error is error 98.13.
`apply(f, list)` (class)	Calls the function F with the items of LIST as its arguments.
`car`, `cdr`, `first`, `rest`	The item and the rest of the list.
`second`, `third`, `[n]`, `at(n)`	An item, counted from 1.
`nthcdr(n)`	The list without its first N items (counted from 0, as in LISP).
`length`, `isList`	The number of cells; 1 if the list ends with `.nil`. A circular list is error 98.12.
`last`, `lastItem`	The last cell, the last item.
`reverse`, `copy`, `append(lists...)`	New lists; `append` shares its last argument, as in LISP.
`member(x)`, `assoc(key)`	The rest of the list starting at X; the first pair whose CAR is KEY.
`map(f)`, `filter(f)`, `reduce(f, initial)`	A list of `f(item)`; the items for which `f(item)` is 1; `f(f(initial, item1), item2)` and so on.
`equal(x)`	True if X has the same structure and equal atoms.
`setCar(x)`, `setCdr(x)`	Changes the cell.
`makeArray`, `string`	An Array of the items; the printed form.

Figure 28. Methods of Cons

`.nil` answers the list methods as the empty list: `.nil~length` is 0, `.nil~car` and `.nil~cdr` are `.nil`, `.nil~map(f)` is `.nil`. `DO x OVER list` walks the items of a list.

A function F is given by its name. It can be an internal routine of the program (for example `square: procedure`), a `::routine`, a built-in function such as `max` or `abs`, or the name of a method that is sent to each item. An internal routine is found in the program that calls `map`, also when the call goes through the `lisp` library.

The lisp library

`::requires 'lisp'` gives the classic LISP functions as routines. The library is built into the interpreter; a file `lisp.cls` next to the program or in REGINA_MACROS replaces it. Its source is in `lib/lisp.cls` of the repository.

Function	Meaning
`cons(a, d)`, `list(items...)`	A new cell; a new list.

Function	Meaning
`car(x)`, `cdr(x)`, `caar`, `cadr`, `cdar`, `cddr`, `caddr`	Parts of a list.
`null(x)`, `atom(x)`, `consp(x)`, `listp(x)`	Tests: the empty list, not a cell, a cell, a list.
`nth(n, l)`, `nthcdr(n, l)`	Counted from 0, as in LISP.
`llength(l)`, `lreverse(l)`, `last(l)`	Length, reverse and last cell; LENGTH and REVERSE are Rexx built-in functions, hence the L.
`append(l...)`, `member(x, l)`, `assoc(k, alist)`	As the methods.
`mapcar(f, l)`, `filter(f, l)`, `reduce(f, l, init)`, `apply(f, l)`	Functions that take a function name.
`equal(a, b)`, `eq(a, b)`	Same structure; the same object or equal atoms.
`rplaca(c, x)`, `rplacd(c, x)`	Change a cell; call them with CALL, as any function whose result is not used.
`read(string)`	A list from its printed form.

Figure 29. Functions of the lisp library

```
::requires 'lisp'
l = list(5, 3, 8)
say cons(1, l) car(l) cdr(l)            /* (1 5 3 8) 5 (3 8) */
say mapcar('square', l) llength(l)      /* (25 9 64) 3       */
say read('(a (b c) . d)')               /* (a (b c) . d)     */
exit
square: procedure; return arg(1) * arg(1)
```

Figure 30. Using the lisp library

Block and the collection methods

Method of Block	Meaning
`new(source)` (class)	A block made from a string, bound to the caller like `{ ... }`.
`call(args...)`, `value(args...)`	Runs the block; returns its result.
`whileTrue(body)`, `whileFalse(body)`	Calls the block, and the block or routine BODY, as long as the block returns 1 (0).
`source`, `string`	The text between the braces; the text with braces.

Figure 31. Methods of Block

The collections Array, List, Queue, Directory, StringTable, Table, Stem and Cons (and `.nil` as the empty list) have these methods. F is a block or the name of a routine, a built-in function or a method, as for the LISP functions. F is called with the item and its index: the position in an Array, List, Queue or LISP list, the key of a Directory or Table, the tail of a Stem.

Method	Result
`do(f)`, `each(f)`	Calls F for every item; no result.
`select(f)`, `reject(f)`	A new collection of the same class with the items for which F returns 1 (0). A Directory keeps the keys.
`collect(f)`	A new collection of the same class with the results of F.
`detect(f [, none])`	The first item for which F returns 1; else NONE (a block is called) or `.nil`.
`inject(initial, f)`	F(F(INITIAL, item1, index1), item2, index2) and so on: sums, maxima, joined strings.
`anySatisfy(f)`, `allSatisfy(f)`, `count(f)`	1 if F is 1 for some (every) item; how many items F accepts.

Method	Result
`sortWith(f)` (Array, List, Queue)	Sorts in place and returns the collection. F(a, b) returns 1 if A may come before B, as in `{ :a :b | a <= b }`. The sort is stable.

Figure 32. Collection methods

A result of F that must be 0 or 1 and is not is error 34.

Python: .Python and .PyObject

Regina OO contains a Python 3 interpreter (the C version of the Python subset interpreter of github.com/mikebeer/python), so a Rexx program can use Python modules: the standard library (math, statistics, json, re, datetime, collections, csv, random, fractions, decimal, itertools and more), NumPy, pandas, SciPy and a matplotlib subset that writes SVG files. Python starts the first time a program uses `.Python` or `.PyObject`; programs that don't use it pay nothing.

`.Python~import(name)` returns a module. Functions, classes and attributes of Python objects are used with messages: `math~sqrt(2)` calls `math.sqrt(2)`, `coll~Counter(words)` creates a Counter, `p~x` reads an attribute and `p~x = 5` sets it. As Rexx message names are in uppercase, the name is looked up as written, then in lowercase, then without regard to case. An attribute that can be called is called with the arguments; any other attribute is returned. `::requires 'python'` is accepted and documents that a program uses Python; it is not required.

Rexx value	Python value
a string that is a number (`42`, `-1.5`, `1E3`)	int or float; whole numbers of any size
any other string	str
`.nil`	None
Array, List, Queue, Cons list	list; an Array with several dimensions as nested lists
Directory, Table, StringTable, Stem object, a stem argument `s.`	dict with str keys
a PyObject	the Python object itself
`.Python~str(s)`, `.Python~bytes(s)`	str or bytes, also when S looks like a number
`.Python~kw(name, value, ...)` as last argument	keyword arguments: `name=value`

Figure 33. Rexx values passed to Python

Python value	Rexx value
None	`.nil`
True, False	1, 0
int, float	the number as a string (`2.5`, `1e-05`, `126765060022822940149670320537 6`)
str, bytes	the string
anything else (list, dict, module, instance, ndarray, DataFrame ...)	a PyObject; `~toRexx` and `~toStem(s.)` convert lists and dicts

Figure 34. Python values returned to Rexx

Method of .Python	Meaning
`import(name)`	The module, also a dotted name such as `'matplotlib.pyplot'`.
`eval(expression [, names])`	The value of a Python expression; NAMES (a Directory or Stem) gives variables.
`exec(statements)`	Runs statements in the main module, for example a `def`; lines are separated by `'0a'x`.
`str(s)`, `bytes(s)`, `kw(name, value, ...)`	See the first table.

Method of .Python	Meaning
any other name	A global of the main module or a built-in: `.python~len(x)`, `.python~sorted(a)`, `.python~repr(x)`, `.python~dict`, `.python~list(a)`.

Figure 35. Methods of .Python

Method of a PyObject	Meaning
any name: `obj~name(args)`, `obj~name = v`	Attribute or method of the Python object.
`[k]`, `at(k...)`, `[k] = v`, `put(v, k...)`	Item access `obj[k]`; several keys make a tuple (`a~at(1, 0)` is `a[1, 0]`). Indexes are those of Python: lists count from 0.
`makeArray`	An Array of the items, used by DO OVER: `do x over pylist`; for a dict the keys.
`toRexx`	Deep copy: lists, tuples and sets become Arrays, dicts and other mappings Directories.
`toStem(s.)`	A mapping fills `s.KEY` (keys in uppercase, as tails of symbols); other items fill `s.1` to `s.n` and `s.0`. Returns the count.
`string`	The `str()` of the object, used by SAY.
`pyType`	The name of the Python type.
`getAttr(n)`, `setAttr(n, v)`, `invoke(n, args...)`, `call(args...)`	An attribute by its exact name, also when its name is a method of Object (`invoke('copy')`); the object called as a function.
`+ - * / % // ** = == \= < > <= >= & | && -x`	Python's operators; Rexx `%` is `//` in Python and Rexx `//` is `%`.

Figure 36. Methods of PyObject

A Python exception is error 98.14 with the Python message, for example `Error 98.14: [Python: ValueError: math domain error]`; `SIGNAL ON SYNTAX` traps it. Output of Python's `print` appears in order with the output of SAY.

```
::requires 'python'
math = .python~import('math')
stats = .python~import('statistics')
say math~sqrt(2)                              /* 1.4142135623730951 */
say stats~mean(.array~of(1, 2, 3, 4))         /* 2.5 */
np = .python~import('numpy')
a = np~array(.array~of(1, 2, 3))
say a * 2 + 1  (a * a)~sum                    /* [3 5 7] 14 */
json = .python~import('json')
d = json~loads('{"city": "Bern", "zip": 3000}')
d~toStem(addr.)
say addr.city addr.zip                        /* Bern 3000 */
.python~print('a', 'b', .python~kw('sep', '-'))  /* a-b */
```

Figure 37. Using Python modules

Where Python finds modules

Modules are searched in the folder of the Rexx program (so a file `mymod.py` next to the program is imported with `.python~import('mymod')`), in the folders of the environment variable PYLIB, in a folder `pylib` next to the rexx executable, and in `share/regina-rexx/pylib` of the installation (`/usr/share/regina-rexx/pylib` for the Debian package). The packages contain the library; in the source tree it is in `python/pylib`.

Chapter 8. Sample Programs

The sample programs are in the folder `demo/oo` of every package and of the repository; the Debian package installs them in `/usr/share/doc/regina-rexx-oo/examples`. Each section below shows a program and its output. The output is checked by the `samples` test suite, so it is what the program prints.

Program	Shows
`01_classes.rexx`	Classes and methods
`02_inheritance.rexx`	Inheritance and class libraries
`03_collections.rexx`	Collections
`04_operators.rexx`	Operator methods
`05_stems.rexx`	Stems as arguments and results
`06_data_structures.rexx`	Data structures
`07_dynamic.rexx`	Dynamic messages
`08_inventory.rexx`	A small application
`09_lisp.rexx`	LISP-style lists
`10_python.rexx`	Python libraries
`11_blocks.rexx`	Blocks
`shapes.cls`	The class library used by 02_inheritance

Figure 38. The sample programs

Classes and methods

A bank account class shows instance variables, read-only attributes, a private helper method, class methods that count the accounts, and a STRING method that SAY uses.

```
/* 01_classes.rexx - classes, attributes, methods, class methods
 *
 * A bank account class: instance variables (EXPOSE), read-only attributes,
 * a private helper method, a class method counting the accounts, and a
 * STRING method that SAY uses.
 */
a = .account~new('Alice', 100)
b = .account~new('Bob')
a~deposit(50)
b~deposit(20)
a~transfer(30, b)
say a                                /* SAY calls the STRING method */
say b
say 'accounts opened:' .account~count

if \b~withdraw(500) then
  say 'refused: Bob has only' b~balance
say 'owner is read-only, balance is' a~balance
exit

::class Account
::method count class                 /* class methods have their */
  expose n                           /* own variables            */
  if symbol('N') <> 'VAR' then n = 0
  return n

::method opened class
```

```
  expose n
  if symbol('N') <> 'VAR' then n = 0
  n = n + 1

::attribute owner get                  /* getters only */
::attribute balance get

::method init                          /* runs for every ~new */
  expose owner balance
  use arg owner, balance
  if arg(2, 'O') then balance = 0
  self~class~opened

::method deposit
  expose balance
  use arg amount
  if self~valid(amount) then balance = balance + amount

::method withdraw                      /* returns .true or .false */
  expose balance
  use arg amount
  if \self~valid(amount) | amount > balance then return .false
  balance = balance - amount
  return .true

::method transfer
  use arg amount, other
  if self~withdraw(amount) then other~deposit(amount)

::method valid private                 /* only callable via self */
  use arg amount
  return datatype(amount, 'N') & amount > 0

::method string
  expose owner balance
  return left(owner, 8) right(format(balance, , 2), 10)
```

Figure 39. 01_classes.rexx

```
rexx 01_classes.rexx
Alice        120.00
Bob           50.00
accounts opened: 2
refused: Bob has only 50
owner is read-only, balance is 120
```

Figure 40. Output of 01_classes.rexx

Inheritance and class libraries

Shapes with an abstract AREA method, defined in the library `shapes.cls` and loaded with `::requires`. The loop handles every shape the same way (polymorphism).

```
/* 02_inheritance.rexx - inheritance, polymorphism, ::requires
 *
 * The classes live in shapes.cls. Every shape answers AREA in its own
 * way; the loop doesn't care which kind of shape it has.
 */
::requires 'shapes.cls'                 /* load the class library */

shapes = .array~of(.circle~new(1), .rectangle~new(2, 3), .square~new(4))
```

```
total = 0
do s over shapes
  say s~describe
  total = total + s~area
end
say 'total area' format(total, , 2)

sq = shapes[3]
say 'a square is a rectangle:' sq~isA(.rectangle)
say 'and a shape:' sq~isA(.shape) ' class:' sq~class~id
say 'superclass of Square:' .square~superclass~id

signal on syntax name abstract
x = .shape~new('blob')~area             /* Shape doesn't implement AREA */
exit
abstract:
  say 'as expected:' condition('D')
```

Figure 41. 02_inheritance.rexx

```
/* shapes.cls - a small class library, loaded with ::requires 'shapes.cls'
 * Shape is abstract: every subclass must implement AREA.
 */
::class Shape public
::attribute name get
::method init
  expose name
  use arg name
::method area abstract
::method describe
  return left(self~name, 10) 'area' format(self~area, , 2)

::class Circle subclass Shape public
::constant pi 3.14159265
::method init
  expose r
  use arg r
  self~init:super('circle')          /* run Shape's INIT too */
::method area
  expose r
  return self~pi * r * r

::class Rectangle subclass Shape public
::method init
  expose w h
  use arg w, h, name
  if arg(3, 'O') then name = 'rectangle'
  self~init:super(name)
::method area
  expose w h
  return w * h

::class Square subclass Rectangle public
::method init
  use arg side
  self~init:super(side, side, 'square')
```

Figure 42. shapes.cls

```
rexx 02_inheritance.rexx
circle     area 3.14
rectangle  area 6.00
```

```
square     area 16.00
total area 25.14
a square is a rectangle: 1
and a shape: 1  class: SQUARE
superclass of Square: RECTANGLE
as expected: Error 98.6: [Method "AREA" is abstract]
```

Figure 43. Output of 02_inheritance.rexx

Collections

Counting words with a Directory, sorting with an Array, a queue of jobs, objects as keys in a Table, and a Supplier.

```
/* 03_collections.rexx - Array, Directory, Queue, Table, Supplier
 *
 * Counts the words of a text with a Directory, sorts them with an Array,
 * walks a queue of jobs, keeps objects as keys in a Table and fills a
 * two-dimensional array.
 */
text = 'the quick brown fox jumps over the lazy dog the fox sleeps'

/* word frequencies: Directory index -> count */
counts = .directory~new
do w over text~makeArray(' ')          /* a string split into an Array */
  if counts~hasIndex(w) then counts[w] = counts[w] + 1
  else counts[w] = 1
end
say counts~items 'different words'

/* sort "count word" pairs; right() makes the counts sort as text */
pairs = .array~new
do with index w item n over counts
  pairs~append(right(n, 3) w)
end
pairs~sort
say 'most frequent:'
do i = pairs~items to pairs~items - 2 by -1
  parse value pairs[i] with n w
  say '  ' left(w, 6) n
end

/* a queue of jobs: first in, first out */
jobs = .queue~new
jobs~queue('print report')
jobs~queue('send mail')
jobs~push('urgent: backup')            /* PUSH puts it in front */
do while \jobs~isEmpty
  say 'doing' jobs~pull
end

/* a Table accepts objects as indexes */
alice = .directory~new; alice['name'] = 'Alice'
bob = .directory~new;   bob['name'] = 'Bob'
manager = .table~new
manager[bob] = alice
say bob['name'] 'reports to' manager[bob]['name']

/* a Supplier walks index/item pairs */
s = .array~of('red', 'green', 'blue')~supplier
do while s~available
  say s~index':' s~item
```

```
  s~next
end

/* an array with two dimensions: a small multiplication table */
t = .array~new(3, 3)
do i = 1 to 3
  do j = 1 to 3
    t[i, j] = i * j
  end
end
say 'table' t~dimensions~toString(, 'x')':'
do i = 1 to t~dimension(1)
  say '  ' right(t[i, 1], 2) right(t[i, 2], 2) right(t[i, 3], 2)
end
```

Figure 44. 03_collections.rexx

```
rexx 03_collections.rexx
9 different words
most frequent:
   the    3
   fox    2
   sleeps 1
doing urgent: backup
doing print report
doing send mail
Bob reports to Alice
1: red
2: green
3: blue
table 3x3:
    1  2  3
    2  4  6
    3  6  9
```

Figure 45. Output of 03_collections.rexx

Operator methods

A Fraction class that defines `+ - * / = <` and the prefix minus, so fractions calculate exactly like numbers.

```
/* 04_operators.rexx - operator methods
 *
 * A Fraction class that defines + - * / = < and the prefix minus.
 * When the left operand of an operator is an object, Rexx sends the
 * operator as a message to it, so a + b means a~"+"(b).
 */
a = .fraction~new(1, 2)
b = .fraction~new(1, 3)
say a '+' b '=' a + b
say a '-' b '=' a - b
say a '*' b '=' a * b
say a '/' b '=' a / b
say '-('a') =' (-a)                  /* prefix minus: a~"-"() */
say a '+ 1 =' a + 1                    /* plain numbers work too */
say 'a = 2/4 ?' (a = .fraction~new(2, 4))
say 'b < a ?' (b < a)

/* sum of 1/1 + 1/2 + ... + 1/6, exactly */
sum = .fraction~new(0)
do n = 1 to 6
```

```
  sum = sum + .fraction~new(1, n)
end
say 'harmonic(6) =' sum '=' format(sum~value, , 5)
exit

::class Fraction
::method init                          /* stored in lowest terms */
  expose num den
  use arg num, den
  if arg(2, 'O') then den = 1
  if den < 0 then do; num = -num; den = -den; end
  g = gcd(abs(num), den)
  num = num % g; den = den % g
::method num;   expose num; return num
::method den;   expose den; return den
::method value; expose num den; return num / den

::method "+"
  expose num den
  use arg o
  o = asFraction(o)
  return .fraction~new(num * o~den + o~num * den, den * o~den)
::method "-"
  expose num den
  if arg() = 0 then return .fraction~new(-num, den)    /* prefix minus */
  use arg o
  o = asFraction(o)
  return .fraction~new(num * o~den - o~num * den, den * o~den)
::method "*"
  expose num den
  use arg o
  o = asFraction(o)
  return .fraction~new(num * o~num, den * o~den)
::method "/"
  expose num den
  use arg o
  o = asFraction(o)
  return .fraction~new(num * o~den, den * o~num)
::method "="
  expose num den
  use arg o
  o = asFraction(o)
  return num = o~num & den = o~den
::method "<"
  expose num den
  use arg o
  o = asFraction(o)
  return num * o~den < o~num * den
::method string
  expose num den
  if den = 1 then return num
  return num'/'den

::routine asFraction                   /* accept 3 as well as 3/1 */
  use arg x
  if x~isA(.fraction) then return x
  return .fraction~new(x)

::routine gcd
  use arg a, b
  do while b > 0
    parse value b a // b with a b
```

```
  end
  if a = 0 then return 1
  return a
```

Figure 46. 04_operators.rexx

```
rexx 04_operators.rexx
1/2 + 1/3 = 5/6
1/2 - 1/3 = 1/6
1/2 * 1/3 = 1/6
1/2 / 1/3 = 3/2
-(1/2) = -1/2
1/2 + 1 = 3/2
a = 2/4 ? 1
b < a ? 1
harmonic(6) = 49/20 = 2.45000
```

Figure 47. Output of 04_operators.rexx

Stems as arguments and results

A routine that sorts the caller's stem in place, a routine that returns a new stem, and a returned stem used as an object.

```
/* 05_stems.rexx - stems by reference, stems as results, Stem objects
 *
 * Classic Rexx passes a stem as the string "NAME.", so routines usually
 * EXPOSE global stems. With the OO extension, USE ARG s. binds the
 * caller's stem directly, and RETURN x. gives a whole stem back.
 * The marker below switches the OO keywords on for this file.
 */
/* OOREGINA */
names.0 = 4
names.1 = 'Wilma'; names.2 = 'Fred'; names.3 = 'Barney'; names.4 = 'Betty'

call sortstem names.                    /* sorted in place */
do i = 1 to names.0
  say i names.i
end

long. = filter(names., 5)               /* a new stem comes back */
say long.0 'names longer than 5:' long.1

call addtotals names., 'COUNT'     /* tails are case sensitive */
say 'added by the routine:' names.count

/* a returned stem kept in a variable is a Stem object */
s = filter(names., 0)
say 'items' s~items '- first' s[1] '- tails' s~allIndexes~toString(, ' ')
do with index t item v over s
  say '  'left(t, 6) v
end
exit

sortstem: procedure                     /* a simple insertion sort */
  use arg s.                            /* s. is the caller's stem */
  do i = 2 to s.0
    v = s.i
    do j = i - 1 to 1 by -1 while s.j > v
      k = j + 1; s.k = s.j
    end
```

```
    k = j + 1; s.k = v
  end
  return

filter: procedure                       /* names longer than min */
  use arg in., min
  out.0 = 0
  do i = 1 to in.0
    if length(in.i) > min then do
      n = out.0 + 1; out.n = in.i; out.0 = n
    end
  end
  return out.

addtotals: procedure
  use arg s., tail
  s.tail = s.0
  return
```

Figure 48. 05_stems.rexx

```
rexx 05_stems.rexx
1 Barney
2 Betty
3 Fred
4 Wilma
1 names longer than 5: Barney
added by the routine: 4
items 5 - first Barney - tails 0 1 2 3 4
  0      4
  1      Barney
  2      Betty
  3      Fred
  4      Wilma
```

Figure 49. Output of 05_stems.rexx

Data structures

A Stack built on Array and a binary search tree of Node objects linked by references.

```
/* 06_data_structures.rexx - building your own data structures
 *
 * A Stack as a subclass of Array, and a binary search tree made of Node
 * objects that refer to each other. .nil marks a missing child.
 */
st = .stack~new
st~push('a'); st~push('b'); st~push('c')
say 'stack top' st~peek', size' st~items
say 'popped' st~pop st~pop', left' st~items

tree = .tree~new
do w over 'mango apple peach banana cherry apple kiwi'~makeArray(' ')
  tree~insert(w)
end
say 'tree holds' tree~count 'distinct words'
say 'in order :' tree~inorder~toString(, ' ')
say 'has kiwi :' tree~contains('kiwi')
say 'has lemon:' tree~contains('lemon')
say 'depth    :' tree~depth
exit
```

```
::class Stack subclass Array           /* inherits APPEND, ITEMS, ... */
::method push
  use arg item
  self~append(item)
::method pop
  if self~items = 0 then return .nil
  return self~remove(self~last)
::method peek
  if self~items = 0 then return .nil
  return self[self~last]

::class Node
::attribute value
::attribute left
::attribute right
::method init
  expose value left right
  use arg value
  left = .nil; right = .nil

::class Tree
::method init
  expose root count
  root = .nil; count = 0
::attribute count get

::method insert
  expose root count
  use arg v
  if root~isNil then do
    root = .node~new(v); count = 1; return
  end
  n = root
  do forever
    select
      when v == n~value then return            /* already there */
      when v << n~value then do
        if n~left~isNil then do; n~left = .node~new(v); leave; end
        n = n~left
      end
      otherwise
        if n~right~isNil then do; n~right = .node~new(v); leave; end
        n = n~right
    end
  end
  count = count + 1

::method contains
  expose root
  use arg v
  n = root
  do while \n~isNil
    if v == n~value then return .true
    if v << n~value then n = n~left
    else n = n~right
  end
  return .false

::method inorder                       /* sorted items, recursively */
  expose root
  list = .array~new
```

```
  call walk root, list
  return list

::method depth
  expose root
  return height(root)

::routine walk
  use arg n, list
  if n~isNil then return
  call walk n~left, list
  list~append(n~value)
  call walk n~right, list

::routine height
  use arg n
  if n~isNil then return 0
  return 1 + max(height(n~left), height(n~right))
```

Figure 50. 06_data_structures.rexx

```
rexx 06_data_structures.rexx
stack top c, size 3
popped c b, left 1
tree holds 6 distinct words
in order : apple banana cherry kiwi mango peach
has kiwi : 1
has lemon: 0
depth    : 5
```

Figure 51. Output of 06_data_structures.rexx

Dynamic messages

A logging proxy built with UNKNOWN, a mixin class that adds PRINT to two unrelated classes, and messages chosen at run time with SEND and SENDWITH.

```
/* 07_dynamic.rexx - UNKNOWN, mixins, messages chosen at run time
 *
 * A logging proxy forwards every message it doesn't know to another
 * object (UNKNOWN). A mixin adds the same methods to unrelated classes
 * (INHERIT). SEND and SENDWITH send a message whose name is in a variable.
 */
list = .array~new
log = .logger~new(list)               /* the proxy wraps the Array */
log~append('first')
log~append('second')
say 'proxied array has' log~items 'items:' log~toString(, ', ')

/* a mixin: Printable adds PRINT to two unrelated classes */
.point~new(3, 4)~print
.person~new('Ada', 36)~print
say 'Person has PRINT:' .person~new('x', 0)~hasMethod('print')
say 'Printable is a mixin:' .printable~queryMixinClass
say 'Person inherits it:' .person~isSubclassOf(.printable)

/* message names chosen at run time */
p = .point~new(6, 8)
do m over .array~of('x', 'y', 'length')
  say 'p~'m '=' p~send(m)
end
```

```
say 'scaled:' p~sendWith('scale', .array~of(10))
exit

::class Logger
::method init
  expose target
  use arg target
::method unknown                        /* any message not defined here */
  expose target
  use arg name, args
  say '  [log]' name 'with' args~items 'argument(s)'
  return target~sendWith(name, args)

::class Printable mixinclass Object
::method print
  say self~class~id':' self~describe

::class Point inherit Printable
::method init
  expose x y
  use arg x, y
::method x;      expose x; return x
::method y;      expose y; return y
::method length; expose x y; return sqrt(x * x + y * y)
::method scale
  expose x y
  use arg k
  return .point~new(x * k, y * k)
::method describe
  expose x y
  return '(' || x || ', ' || y || ')'   /* not '('x: that is a hex string */
::method string
  return self~describe

::class Person inherit Printable
::method init
  expose name age
  use arg name, age
::method describe
  expose name age
  return name 'aged' age

::routine sqrt                          /* Newton's method */
  use arg n
  if n = 0 then return 0
  r = n
  do 20
    r = (r + n / r) / 2
  end
  return format(r, , 4) / 1             /* / 1 drops trailing zeros */
```

Figure 52. 07_dynamic.rexx

```
rexx 07_dynamic.rexx
  [log] APPEND with 1 argument(s)
  [log] APPEND with 1 argument(s)
  [log] ITEMS with 0 argument(s)
  [log] TOSTRING with 1 argument(s)
proxied array has 2 items: first, second
POINT: (3, 4)
PERSON: Ada aged 36
Person has PRINT: 1
```

```
Printable is a mixin: 1
Person inherits it: 1
p~x = 6
p~y = 8
p~length = 10
scaled: (60, 80)
```

Figure 53. Output of 07_dynamic.rexx

A small application

Stock keeping with Product and Inventory classes: orders, a sorted value report and a reorder list.

```
/* 08_inventory.rexx - a small application: stock keeping
 *
 * Products are objects, the inventory is a class holding a Directory of
 * products by article number. Orders change the stock; a report lists
 * the stock value, sorted, and the articles to reorder.
 */
data = 'A100 Screwdriver 4.50 120 50;',
       'A200 Hammer      12.90  15 20;',
       'B310 Drill       89.00   4  5;',
       'B320 Saw         24.00  30 10;',
       'C900 Toolbox     35.50   8  5'

inv = .inventory~new
do line over data~makeArray(';')
  parse var line id name price qty minimum .
  inv~add(.product~new(id, name, price, qty, minimum))
end

orders = .array~of('A200 10', 'B310 3', 'C900 9', 'X999 1', 'A100 25')
do o over orders
  parse var o id n
  say left(o, 10) '->' inv~order(id, n)
end
say
call report inv
exit

report: procedure
  use arg inv
  say 'Article Name          Qty    Value'
  do p over inv~byValue
    say left(p~id, 7) left(p~name, 12) right(p~qty, 5) right(format(p~value, , 2), 8)
  end
  say copies('-', 35)
  say left('Total', 25) right(format(inv~totalValue, , 2), 10)
  reorder = inv~below
  if reorder~items > 0 then
    say 'Reorder:' reorder~toString(, ', ')
  return

::class Product
::attribute id get
::attribute name get
::attribute price get
::attribute qty get
::attribute minimum get
::method init
  expose id name price qty minimum
  use arg id, name, price, qty, minimum
```

```
::method take                          /* returns .false if not enough */
  expose qty
  use arg n
  if n > qty then return .false
  qty = qty - n
  return .true
::method value
  expose qty price
  return qty * price
::method low
  expose qty minimum
  return qty < minimum

::class Inventory
::method init
  expose items
  items = .directory~new
::method add
  expose items
  use arg p
  items[p~id] = p
::method order
  expose items
  use arg id, n
  if \items~hasIndex(id) then return 'unknown article'
  p = items[id]
  if \p~take(n) then return 'only' p~qty 'in stock'
  return 'ok, left' p~qty
::method totalValue
  expose items
  t = 0
  do p over items~allItems
    t = t + p~value
  end
  return t
::method below                         /* names of articles to reorder */
  expose items
  list = .array~new
  do with index id item p over items
    if p~low then list~append(p~name)
  end
  return list
::method byValue                       /* products, highest value first */
  expose items
  keys = .array~new
  do with index id item p over items
    keys~append(right(format(p~value, , 2), 12) id)
  end
  keys~sort
  sorted = .array~new                  /* not "result": a message   */
  do i = keys~items to 1 by -1         /* instruction sets RESULT   */
    sorted~append(items[word(keys[i], 2)])
  end
  return sorted
```

Figure 54. 08_inventory.rexx

```
rexx 08_inventory.rexx
A200 10    -> ok, left 5
B310 3     -> ok, left 1
C900 9     -> only 8 in stock
X999 1     -> unknown article
```

```
A100 25    -> ok, left 95

Article Name          Qty    Value
B320    Saw             30   720.00
A100    Screwdriver     95   427.50
C900    Toolbox          8   284.00
B310    Drill            1    89.00
A200    Hammer           5    64.50
---------------------------------
Total                        1585.00
Reorder: Hammer, Drill
```

Figure 55. Output of 08_inventory.rexx

LISP-style lists

An evaluator for expressions in LISP notation such as `(* 2 (+ 3 4))`, quicksort and flatten written recursively, and an association list.

```
/* 09_lisp.rexx - LISP-style list processing
 *
 * ::requires 'lisp' gives the LISP functions: cons, car, cdr, list,
 * null, atom, append, mapcar, ... Lists are made of Cons cells and .nil
 * is the empty list. The program reads expressions written in LISP
 * notation, evaluates them, and shows recursive list functions.
 */
::requires 'lisp'

/* expressions in prefix notation, read from strings */
do e over .array~of('(+ 1 2 3)', '(* 2 (+ 3 4))', '(- 10 (/ 9 3) 1)', '(max 4 (* 2 5) 7)')
  x = read(e)
  say left(x, 22) '=' evaluate(x)
end

/* recursive list functions in the style of LISP */
l = list(5, 3, 8, 1, 9, 2)
say 'list     ' l
say 'sorted   ' qsort(l)
say 'sum      ' sum(l)
say 'squares  ' mapcar('square', l)
say 'evens    ' filter('even', l)
say 'flattened' flatten(read('(1 (2 (3 4)) ((5) 6))'))

/* an association list: a small phone book */
book = list(cons('alice', 4711), cons('bob', 1234), cons('carol', 5555))
say 'bob has  ' cdr(assoc('bob', book))
exit

evaluate: procedure            /* (op arg1 arg2 ...) or a number */
  use arg x
  if atom(x) then return x
  op = car(x)
  args = mapcar('evaluate', cdr(x))
  select
    when op = '+' then return reduce('plus', args, 0)
    when op = '*' then return reduce('times', args, 1)
    when op = '-' then return reduce('minus', args)
    when op = '/' then return reduce('divide', args)
    otherwise return apply(op, args)       /* any Rexx function: max, min, ... */
  end
```

```
qsort: procedure              /* quicksort on lists */
  use arg l
  if null(l) then return .nil
  p = car(l)
  smaller = .nil; larger = .nil
  do x over cdr(l)
    if x < p then smaller = cons(x, smaller)
    else larger = cons(x, larger)
  end
  return append(qsort(smaller), list(p), qsort(larger))

sum: procedure                /* plain recursion */
  use arg l
  if null(l) then return 0
  return car(l) + sum(cdr(l))

flatten: procedure
  use arg x
  if null(x) then return .nil
  if atom(x) then return list(x)
  return append(flatten(car(x)), flatten(cdr(x)))

square: procedure; return arg(1) * arg(1)
even:   procedure; return arg(1) // 2 = 0
plus:   procedure; return arg(1) + arg(2)
times:  procedure; return arg(1) * arg(2)
minus:  procedure; return arg(1) - arg(2)
divide: procedure; return arg(1) / arg(2)
```

Figure 56. 09_lisp.rexx

```
rexx 09_lisp.rexx
(+ 1 2 3)             = 6
(* 2 (+ 3 4))        = 14
(- 10 (/ 9 3) 1)      = 6
(max 4 (* 2 5) 7)     = 10
list      (5 3 8 1 9 2)
sorted    (1 2 3 5 8 9)
sum       28
squares   (25 9 64 1 81 4)
evens     (8 2)
flattened (1 2 3 4 5 6)
bob has   1234
```

Figure 57. Output of 09_lisp.rexx

Python libraries

Sales figures analysed with the Python modules statistics, collections and json: mean, median and standard deviation, a Counter, totals in a defaultdict read back as a stem, JSON text, and a Python function defined from Rexx.

```
/* 10_python.rexx - Python libraries from Rexx
 *
 * .Python~import loads a Python module; its functions and classes are
 * used with ~ like Rexx methods. Rexx numbers, strings, Arrays and
 * Directories are converted on the way in; Python numbers and strings
 * come back as Rexx strings, everything else as a PyObject.
 * The program analyses a small table of sales figures.
 */
::requires 'python'
```

```
stats = .python~import('statistics')
json = .python~import('json')
coll = .python~import('collections')

/* the data: one Rexx Directory per sale */
sales = .array~new
call sale 'Zurich', 'tea', 120
call sale 'Bern', 'coffee', 340
call sale 'Zurich', 'coffee', 210
call sale 'Basel', 'tea', 90
call sale 'Bern', 'tea', 150
call sale 'Zurich', 'cocoa', 60

amounts = .array~new
do s over sales
  amounts~append(s['amount'])
end
say 'sales     ' amounts~makestring('line', ' ')
say 'total     ' .python~sum(amounts)
say 'mean      ' stats~mean(amounts)
say 'median    ' stats~median(amounts)
say 'stdev     ' format(stats~stdev(amounts), , 2)

/* a Counter of the products, most common first */
products = .array~new
do s over sales
  products~append(s['product'])
end
do pair over coll~Counter(products)~most_common
  say 'product   ' left(pair~at(0), 8) pair~at(1)
end

/* totals per city in a Python dict, back as a Rexx stem */
totals = coll~defaultdict(.python~eval('int'))
do s over sales
  totals~put(totals~at(s['city']) + s['amount'], s['city'])
end
totals~toStem(t.)
say 'Zurich    ' t.zurich
say 'by city   ' json~dumps(.python~dict(totals), .python~kw('sort_keys', 1))

/* the sales as JSON text and back */
text = json~dumps(sales)
say 'json      ' left(text, 60)'...'
again = json~loads(text)~toRexx
say 'read back ' again~items 'sales, first in' again[1]['city']

/* a Python function, defined from Rexx */
.python~exec('def vat(x, rate=8.1):'                    || '0a'x ||,
             '    return round(x * (1 + rate / 100), 2)')
say 'with VAT  ' .python~vat(120) .python~vat(120, .python~kw('rate', 2.6))
exit

sale: procedure expose sales
  d = .directory~new
  d['city'] = arg(1); d['product'] = arg(2); d['amount'] = arg(3)
  sales~append(d)
  return
```

Figure 58. 10_python.rexx

```
rexx 10_python.rexx
sales      120 340 210 90 150 60
total      970
mean       161.66666666666666
median     135.0
stdev      101.47
product    tea      3
product    coffee   2
product    cocoa    1
Zurich     390
by city    {"Basel": 90, "Bern": 490, "Zurich": 390}
json       [{"city": "Zurich", "product": "tea", "amount": 120}, {"city...
read back  6 sales, first in Zurich
with VAT   129.72 123.12
```

Figure 59. Output of 10_python.rexx

Blocks

A small book list processed with blocks: select, collect, inject, count, anySatisfy, detect, reject with a variable of the program, sortWith, a counter that keeps its state in a block, and a block with several clauses.

```
/* 11_blocks.rexx - blocks and the collection methods
 *
 * A block { ... } is a small routine written where it is used.
 * { :x | expression } takes the parameter x and returns the expression;
 * { use arg x; ... } is the long form. Blocks see the variables of the
 * code that created it, and keep them when that code has returned.
 * Collections answer do, select, reject, collect, detect, inject,
 * anySatisfy, allSatisfy, count and (Arrays) sortWith.
 */
books = .array~new
call book 'Dune', 'Herbert', 1965, 412
call book 'Neuromancer', 'Gibson', 1984, 271
call book 'Foundation', 'Asimov', 1951, 255
call book 'Hyperion', 'Simmons', 1989, 482
call book 'Solaris', 'Lem', 1961, 204

say 'Before 1970:'
books~select({ :b | b['year'] < 1970 })~do({ use arg b; call show b })

say 'Titles:    ' books~collect({ :b | b['title'] })~makestring('line', ', ')
say 'Pages:     ' books~inject(0, { :sum :b | sum + b['pages'] })
say 'Long ones: ' books~count({ :b | b['pages'] > 400 })
say 'Any by Lem?' books~anySatisfy({ :b | b['author'] = 'Lem' })
first = books~detect({ :b | b['year'] > 1980 })
say 'First after 1980:' first['title']

limit = 260                                  /* blocks see this variable */
short = books~reject({ :b | b['pages'] > limit })
say 'Short:     ' short~collect({ :b | b['title'] })~makestring('line', ', ')

say 'By year:'
books~sortWith({ :a :b | a['year'] <= b['year'] })~do({ use arg b; call show b })

/* a block that keeps its own state: a counter made by a routine */
next = counter(100)
say 'Tickets:   ' next~call next~call next~call

/* the long form, with several clauses */
stats = { use arg list
          total = list~inject(0, { :s :x | s + x })
```

```
          return 'n='list~items 'mean='format(total / list~items, , 1) }
say 'Stats:     ' stats~call(books~collect({ :b | b['pages'] }))
exit

book: procedure expose books
  d = .directory~new
  d['title'] = arg(1); d['author'] = arg(2); d['year'] = arg(3); d['pages'] = arg(4)
  books~append(d)
  return

show: procedure
  use arg b
  say '  ' left(b['title'], 12) left(b['author'], 8) b['year']
  return

counter: procedure
  use arg n
  return { n = n + 1; return n }
```

Figure 60. 11_blocks.rexx

```
rexx 11_blocks.rexx
Before 1970:
   Dune         Herbert  1965
   Foundation   Asimov   1951
   Solaris      Lem      1961
Titles:     Dune, Neuromancer, Foundation, Hyperion, Solaris
Pages:      1624
Long ones:  2
Any by Lem? 1
First after 1980: Neuromancer
Short:      Foundation, Solaris
By year:
   Foundation   Asimov   1951
   Solaris      Lem      1961
   Dune         Herbert  1965
   Neuromancer  Gibson   1984
   Hyperion     Simmons  1989
Tickets:    101 102 103
Stats:      n=5 mean=324.8
```

Figure 61. Output of 11_blocks.rexx

Chapter 9. Restrictions and Known Problems

Memory

- **No garbage collection.** An object lives until the program ends, even when no variable refers to it any more. A program that creates many short-lived objects in a long loop uses more and more memory. Reuse objects in such loops, or keep the work in strings and stems.
- All objects of a program are freed when the program ends, also when it is called through the REXX API.

Language

- The ooRexx features listed in Chapter 5 under "Not available" are missing, notably RAISE, FORWARD, GUARD, REPLY, START and many classes.
- An array can have up to 64 dimensions and at most 100,000,000 places in all (error 88.6).
- A function given by name to `map`, `filter`, `reduce`, `mapcar` or `apply` must be an internal routine, a `::routine`, a built-in function or a method; external routines are not searched. LISP lists use the CAR/CDR recursion of LISP, so very long lists are better handled with `DO x OVER` than with deep recursion.
- `sort` orders the string values of the items; use `sortWith` with a block for other orders.
- The new keywords need a marker, a directive, an earlier message operator or `--oo` (Chapter 3). A program that uses objects only through routine arguments, without any of these, must add a marker to use them.
- Directives are not allowed in INTERPRET data (error 98.2).
- Only a plain stem argument (`st.`) is passed by reference; a compound variable or an expression is passed by value. `RETURN x.` returns a copy, and `EXIT x.` returns the stem's string value as in classic REXX.
- An operator with an object as right operand and a string as left operand uses the object's string value; the object's operator method is not called.
- Class IDs of classes defined with `::class` are in uppercase.
- PUBLIC and PRIVATE on `::class` and `::routine` are accepted but have no effect: all classes and routines of a file loaded with `::requires` are visible to the program that loads it.
- The collection of a DO OVER loop is copied when the loop starts; changes in the body do not affect the loop.
- In a block only the names of a first `USE ARG` are parameters; `ARG` and `PARSE ARG` in a block set variables of the creator if it has them.
- A block keeps the variables of its creator as a copy made when the creator returns; two routines cannot share variables through a block after both have returned.
- Objects belong to the thread that created them and cannot be shared between threads that run REXX programs at the same time.

Python

- The built-in Python is a subset of Python 3 (see the manual of github.com/mikebeer/python): strings are byte strings, there is no `async`, and the libraries are pure-Python versions of the standard library, NumPy, pandas and SciPy. C extension modules of CPython cannot be loaded.
- Python cannot call back into Rexx: a Rexx routine or block cannot be passed as a callback (for example `key=` of `sorted`). Write such functions in Python with `.python~exec`.
- A string that looks like a number is passed as a number; use `.python~str(s)` for a string such as a postal code `007`.
- An operator with a PyObject on the right and a Rexx string on the left (`2 * a`) uses the string value of the object; write `a * 2`.
- Python objects referred to by Rexx live until the program ends, like all objects.
- There is one Python interpreter per process; it is not shared safely between threads.

Errors and tracing

- An error in a file loaded by `::requires` names the main program in the error message, not the required file.
- TRACE I shows the hidden position counter of DO OVER loops.
- Deep recursion of methods ends with error 11 (control stack full), based on the measured stack use. The Windows and macOS builds have a 256 MB stack.

Platforms

Package	Status
Linux x86-64 (`.deb`)	Built and tested natively: all four test suites, valgrind.
Windows 64-bit (zip)	Cross-compiled with MinGW-w64. The oo test suite passes under Wine; not yet tested on Windows itself. No readline, curses or GCI.
macOS (tar.gz)	Cross-compiled with zig for Apple Silicon and Intel, macOS 11 or later. The same sources and configuration pass all test suites on Linux, but the binary has not yet been run on a Mac. It is ad-hoc signed, not notarized (see Chapter 2). No readline, curses or GCI.

Figure 62. Platform status

Performance

Classic programs run at the speed of Regina 3.9.7: built with the same compiler options, the REXXCPS benchmark gave a median of 9.67 million clauses per second against 9.51 million for Regina 3.9.7 (seven runs each), a difference within the noise of the measurement. A message is slower than a call of an internal routine, because it creates a new variable scope and looks up the method. Collections are implemented in C and are fast; a loop over thousands of items is no problem.

Appendix A. Error Messages

The messages of the extension, as printed by the English message file. "..." stands for a name or value filled in at run time. The texts are also available in German; the other languages show the English texts.

Error	Message
88.0	Invalid argument
88.1	Method "...": argument n must be a positive whole number; found "..."
88.2	Method "...": argument n must be a class object; found "..."
88.3	Method "...": argument n is required
88.4	Method "...": argument n must be an Array; found "..."
88.5	Method "...": argument n must be a non-negative whole number; found "..."
88.6	Array index ... exceeds the maximum of 100000000
91.0	No result object
91.1	Message "..." did not return a result
97.0	Object method not found
97.1	Object "..." does not understand message "..."
98.0	Execution error
98.1	Class "..." not found
98.2	Directives are not allowed in INTERPRET data
98.3	Could not find the required file "..."
98.4	... is only valid in a method
98.5	DO OVER: "..." is not a collection
98.6	Method "..." is abstract
98.7	Mixin class "..." cannot create instances
98.8	Method "..." needs a one-dimensional array
98.9	Supplier has no more items
98.10	Class "..." would inherit from itself
98.11	The array has ... dimension(s), but ... index(es) were given
98.12	The list is circular
98.13	Invalid list: ...
98.14	Python: ...
98.15	Python is not available in this build
98.16	The block is used after the routine that created it has ended; begin the block with PROCEDURE
99.0	Translation error
99.1	Unknown directive "::..."
99.2	Duplicate ::CLASS "..."
99.3	Duplicate method "..." in class "..."
99.4	::... must follow a ::CLASS directive
99.5	Missing name in ::... directive
99.6	Invalid option "..." in ::... directive
99.7	Duplicate ::ROUTINE "..."
99.8	Block not closed: "}" expected

Error	Message
99.9	Invalid block parameters: ":name ..." must be followed by a vertical bar and an expression

Figure 63. Error messages of the extension

Errors of the extension are SYNTAX conditions: `SIGNAL ON SYNTAX` traps them, RC holds the main number and `condition('D')` the message.

Appendix B. Switches and Environment Variables

Switch or variable	Meaning
`--oo`, `-O`	The OO keywords are on in every program, required file and INTERPRET string.
`--no-oo`, `-N`	The extension is off: Regina 3.9.7 behaviour.
`REGINA_OO`	`on`, `1`, `yes` or `oo` act like `--oo`; `off`, `0`, `no` or `classic` like `--no-oo`. A command-line switch wins.
`REGINA_MACROS`	Folders searched for external routines and for `::requires` files.
`REGINA_LANG`, `REGINA_LANG_DIR`	Language of the error messages and the folder of the `*.mtb` files.
`PYLIB`	Folders searched for Python modules, separated by `:` (`;` on Windows), before the library of the installation.

Figure 64. Switches and environment variables

All other switches of Regina 3.9.7 work as before; `rexx -h` lists them.

Appendix C. Contents of the Packages

Package	Contents
Debian	`/usr/bin`: rexx, regina, rxstack, rxqueue, regina-config `/usr/lib/<arch>`: libregina.so, libregina.a, add-on libraries `/usr/share/regina-rexx`: message files, the classic examples and `pylib` (the Python library) `/usr/share/doc/regina-rexx-oo`: README.OO.md, this manual, examples (the OO samples) `/usr/share/man/man1`: man pages
Windows	`rexx.exe`, `*.mtb`, `pylib` (the Python library), `README.txt`, `README.OO.md`, this manual, `COPYING-LIB.txt`, `demo` with the classic examples and `demo\oo` with the OO samples
macOS	`rexx` (universal), `*.mtb`, `pylib`, `INSTALL-macOS.txt`, `README.OO.md`, this manual, `COPYING-LIB.txt`, `demo` and `demo/oo`

Figure 65. Package contents